Model Deployment and Scaling

Model Deployment and Scaling in AI Systems

Building an accurate Machine Learning or Generative AI model is only one part of the AI lifecycle.

The real challenge begins when organizations need to:

- Deploy models into production
- Serve predictions at scale
- Minimize latency
- Optimize infrastructure cost
- Support millions of users
- Ensure reliability and observability

This operational phase is known as:

Model Deployment and Scaling

Modern AI systems must support:

- Real-time inference
- Batch inference
- Edge inference
- Containerized deployment
- Kubernetes orchestration
- Horizontal scaling
- GPU optimization

This chapter explores the complete deployment ecosystem for production AI systems.

Section 1 — Theory & Conceptual Foundation

Chapter 1 — Introduction to AI Inference

1.1 What Is Inference?

Inference refers to:

The process of using a trained AI model to make predictions or generate outputs.

Once a model completes training, it enters the:

Inference Phase

where it processes real-world inputs and produces predictions.

Examples of AI Inference

Application	Inference Output
Chatbot	Generated response
Fraud Detection	Fraud probability
Recommendation System	Suggested products
Medical AI	Disease prediction
Self-driving car	Object detection

Training vs Inference

Phase	Purpose
Training	Learn patterns from data
Inference	Use learned patterns for predictions

Training is computationally expensive but infrequent.

Inference is continuous and production-facing.

Chapter 2 — Batch Inference

2.1 What Is Batch Inference?

Batch inference processes:

Multiple inputs together in batches.

Instead of processing requests individually, the system groups inputs and runs them simultaneously.

2.2 Batch Inference Workflow

The uploaded slides show the following workflow:

Input Collection
↓
Batch Formation
↓
Model Processing
↓
Output Distribution

Step-by-Step Explanation

Step 1 — Input Collection

The system gathers:

- Images
- Text
- Sensor data
- Business records

before inference begins.

Step 2 — Batch Formation

Inputs are grouped into batches.

Example:

1000 customer records

processed together.

Step 3 — Model Processing

The model processes the entire batch simultaneously.

This improves:

- GPU utilization
- Parallel computation
- Throughput efficiency

Step 4 — Output Distribution

Predictions are distributed back to:

- Databases
- APIs
- Analytics systems
- Business workflows

2.3 Advantages of Batch Inference

The uploaded slides highlight key advantages:

Benefit	Explanation
Speed	Efficient parallel processing
Cost efficiency	Better hardware utilization
Throughput	Large-scale prediction capability

Why Batch Inference Is Efficient

Modern GPUs are optimized for:

Parallel Tensor Operations

Batching maximizes GPU efficiency.

2.4 Real-World Batch Inference Use Cases

Batch inference is commonly used for:

- Fraud analysis
- Recommendation generation
- Document classification
- Large-scale embeddings
- Nightly analytics jobs

2.5 Challenges with Batch Inference

The uploaded notes identify several challenges.

Challenge	Description
Memory management	Large batches consume RAM/GPU memory
Variable input sizes	Uneven inputs complicate batching
Latency requirements	Not suitable for immediate responses

Why Latency Becomes a Problem

Batch systems often wait until enough requests accumulate before processing.

This increases:

Inference Delay

making them unsuitable for interactive applications.

Chapter 3 — Real-Time Inference

3.1 What Is Real-Time Inference?

Real-time inference refers to:

Generating predictions immediately—or nearly immediately—after receiving an input.

This is critical for:

- Chatbots
- AI copilots

- Voice assistants
- Fraud detection
- Autonomous systems

3.2 Real-Time Inference Workflow

The uploaded architecture shows:

Input Collection
↓
Preprocessing
↓
Model Inference
↓
Postprocessing
↓
Response Delivery

Step-by-Step Breakdown

Step 1 — Input Collection

The system receives:

- User prompts
- Audio streams
- API requests
- Images

Step 2 — Preprocessing

Input is cleaned and transformed.

Examples:

- Tokenization
- Image resizing

- Normalization

Step 3 — Model Inference

The model generates predictions in real time.

This stage is latency-sensitive.

Step 4 — Postprocessing

The raw model output is refined.

Examples:

- Formatting
- Ranking
- Filtering unsafe content

Step 5 — Response Delivery

The final result is returned to the user or downstream application.

3.3 Real-Time Inference Use Cases

The uploaded slides identify several applications.

Application	Description
Conversational AI	Chatbots and copilots
Live translation	Real-time language conversion
Text-to-speech	AI voice systems
Gaming	Dynamic AI interactions
Streaming/video	Live AI enhancements

Why Real-Time Systems Are Difficult

Real-time AI systems require:

- Extremely low latency
- Fast GPU inference
- Efficient networking
- High availability

Even small delays impact user experience.

Chapter 4 — Optimization Strategies

4.1 Why Optimization Matters

Inference systems often face:

- High traffic
- GPU bottlenecks
- Memory constraints
- Expensive infrastructure

Optimization reduces:

- Latency
- Cost
- Resource consumption

4.2 Major Optimization Strategies

The uploaded material highlights several strategies.

Strategy	Purpose
Model optimization	Reduce model complexity
Hardware acceleration	Use GPUs/TPUs
Dynamic batching	Combine real-time requests efficiently

4.3 Dynamic Batching

Dynamic batching:

- Collects requests briefly
- Combines them dynamically
- Processes together

This improves:

- Throughput
- GPU efficiency
- Cost optimization

while maintaining acceptable latency.

Chapter 5 — Edge Inference

5.1 What Is Edge Inference?

Edge inference refers to:

Running AI models directly on edge devices instead of centralized cloud infrastructure.

Examples of edge devices:

- Smartphones
- IoT sensors
- Drones
- Smart cameras
- Autonomous vehicles

Why Edge Inference Matters

Edge AI provides:

- Lower latency
- Offline capability
- Improved privacy
- Reduced bandwidth usage

5.2 Edge Inference Workflow

The uploaded slides show:

Model Deployment
↓
Input Processing
↓
On-Device Inference
↓
Output Delivery

Step-by-Step Breakdown

Step 1 — Model Deployment

Optimized model is deployed onto edge device.

Step 2 — Input Processing

The device processes local sensor or user input.

Step 3 — On-Device Inference

Predictions occur locally without cloud communication.

Step 4 — Output Delivery

The device immediately acts on predictions.

Examples:

- Object detection
- Voice response
- Sensor automation

Edge AI Challenges

Edge systems have limited:

- Memory
- Compute power
- Battery life

Therefore models require:

- Quantization
- Compression
- Optimization

Chapter 6 — Docker for AI Deployment

6.1 What Is Docker?

The uploaded slides define Docker as:

An open-source platform that uses containerization to simplify application deployment.

Docker packages:

- Code
- Dependencies
- Runtime
- System libraries

into isolated containers.

Why Docker Matters in AI

AI systems often suffer from:

"It works on my machine."

Docker eliminates environment inconsistency.

6.2 Docker Workflow

The uploaded pipeline shows:

Dockerfile
↓
Build Image
↓
Run Container

Step-by-Step Breakdown

Step 1 — Dockerfile

Defines:

- Base image
- Dependencies
- Runtime configuration

Step 2 — Build Image

Creates portable container image.

Step 3 — Run Container

Deploys isolated runtime environment.

Example Dockerfile for FastAPI AI App

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Detailed Explanation

FROM python:3.11

Uses official Python runtime image.

WORKDIR /app

Defines working directory.

COPY requirements.txt .

Copies dependency file.

RUN pip install

Installs required libraries.

COPY . .

Copies application source code.

CMD

Launches inference API.

Chapter 7 — Kubernetes (K8s)

7.1 What Is Kubernetes?

The uploaded slides define Kubernetes as:

An open-source platform for managing containerized applications across clusters of servers.

Kubernetes is commonly abbreviated as:

K8s

Why Kubernetes Matters

AI systems require:

- Scalability
- Fault tolerance
- High availability
- Auto-scaling

Kubernetes solves these operational problems.

7.2 Kubernetes Architecture

The uploaded diagram includes:

- Control Plane
- Nodes
- Pods
- Containers
- Kubelets

Control Plane

The control plane manages:

- Scheduling
- Scaling
- Cluster state
- Deployments

Think of it as:

The Brain of Kubernetes

Nodes

Nodes are worker machines that run workloads.

Each node contains:

- Pods
- Containers
- Kubelet services

Pods

Pods are the smallest deployable units in Kubernetes.

A pod may contain:

- One container
- Multiple tightly coupled containers

Containers

Containers run:

- AI APIs
- Model servers
- Inference pipelines

Kubelet

Kubelet ensures containers remain healthy and operational.

Chapter 8 — Amazon SageMaker

8.1 What Is SageMaker?

Amazon SageMaker is a managed cloud platform for:

- Training
- Optimizing
- Deploying

- Scaling ML models

provided by Amazon Web Services.

8.2 SageMaker Workflow

The uploaded slides show:

Model Selection
↓
Training
↓
Optimization
↓
Deployment

Step-by-Step Explanation

Step 1 — Model Selection

Choose:

- Framework
- Algorithm
- Foundation model

Step 2 — Training

Train models using scalable cloud infrastructure.

Step 3 — Optimization

Optimize:

- Latency
- Throughput
- Cost

Step 4 — Deployment

Deploy inference endpoints.

8.3 SageMaker Features

The uploaded slides identify key features.

Feature	Description
Multi-model support	Multiple models on one endpoint
Real-time inference	Low-latency predictions
Batch inference	Large-scale offline processing

Chapter 9 — Scaling AI Systems

9.1 What Is Horizontal Scaling?

Horizontal scaling refers to:

Adding more instances of applications or model endpoints to handle increased traffic.

Instead of:

One Large Server

organizations deploy:

Many Smaller Servers

9.2 Benefits of Horizontal Scaling

The uploaded notes identify major benefits.

Benefit	Explanation
Load distribution	Traffic spread across instances
Reduced queuing	Faster request handling
Improved parallelism	More concurrent inference

Why Parallelism Matters

AI inference workloads are highly parallelizable.

Horizontal scaling improves:

- Throughput
- Reliability
- Availability

9.3 Why Scaling Alone Does Not Reduce Latency

The uploaded material highlights important limitations.

Problem	Impact
Heavy computational requirements	Slow inference
Inefficient model design	Poor optimization
Data transfers	Network bottlenecks

Example

Even with many servers:

- Large LLMs may remain slow
- GPU communication may bottleneck
- Token generation remains sequential

Scaling solves:

Throughput

more effectively than:

Single-request latency

Section 2 — Practical Implementation & Coding Walkthrough

Chapter 10 — Deploying AI Models with FastAPI and Docker

10.1 Install Dependencies

Cell 1 — Install Required Libraries

```
pip install fastapi uvicorn torch transformers
```

Detailed Explanation

This installs:

- API framework
- ASGI server
- Deep learning framework
- Transformer model library

10.2 Create Inference API

Cell 2 — FastAPI Application

```
from fastapi import FastAPI
from transformers import pipeline

app = FastAPI()

classifier = pipeline(
  "sentiment-analysis"
)

@app.get("/")
def home():
  return {
    "message": "Inference API Running"
  }
```

Detailed Explanation

This creates:

- FastAPI backend
- Hugging Face inference pipeline

- Root endpoint

Why Hugging Face Pipelines Matter

Pipelines simplify:

- Model loading
- Tokenization
- Inference orchestration

10.3 Create Prediction Endpoint

Cell 3 — Real-Time Inference Endpoint

```
@app.post("/predict")
def predict(text: str):

  result = classifier(text)

  return {
    "prediction": result
  }
```

Detailed Explanation

This endpoint:

1. Receives text input
2. Runs inference
3. Returns predictions

This forms:

Real-Time AI Inference API

10.4 Run the API

Cell 4 — Start Server

```
uvicorn app.main:app --reload
```

Detailed Explanation

Starts FastAPI development server.

10.5 Dockerize the AI System

Cell 5 — Dockerfile

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app",
"--host", "0.0.0.0",
"--port", "8000"]
```

Why Containerization Matters

Containers ensure:

- Reproducibility
- Portability
- Scalable deployment

10.6 Build Docker Image

Cell 6 — Build Container

```
docker build -t ai-inference-app .
```

Detailed Explanation

Creates deployable Docker image.

10.7 Run Docker Container

Cell 7 — Start Container

```
docker run -p 8000:8000 ai-inference-app
```

Detailed Explanation

Runs containerized inference API.

10.8 Kubernetes Deployment Example

Cell 8 — Deployment YAML

```
apiVersion: apps/v1
kind: Deployment

metadata:
 name: ai-inference

spec:
 replicas: 3

 selector:
  matchLabels:
   app: ai-inference

 template:
  metadata:
   labels:
    app: ai-inference

  spec:
   containers:
   - name: inference-container
    image: ai-inference-app
    ports:
    - containerPort: 8000
```

Detailed Explanation

This Kubernetes deployment:

- Creates 3 replicas
- Enables horizontal scaling
- Improves availability
- Supports load balancing

Understanding Key Components

---

replicas: 3

Runs three instances of the inference API.

---

containers

Defines deployed AI container.

---

containerPort: 8000

Exposes inference API.

10.9 Production-Level AI Deployment Architecture

Modern AI deployment systems often look like:

User
↓
Load Balancer
↓
API Gateway
↓
Kubernetes Cluster
↓
Inference Pods
↓
GPU Nodes

↓

Monitoring & Logging

This architecture supports:

- Scalability
- Reliability
- Fault tolerance
- Observability

Final Takeaways

Modern AI deployment is no longer:

"Run model.py"

Production AI systems require:

- Inference optimization
- Containerization
- Kubernetes orchestration
- Horizontal scaling
- GPU acceleration
- Real-time APIs
- Monitoring
- Cost optimization

Key deployment paradigms include:

- Batch inference
- Real-time inference
- Edge inference

Key infrastructure technologies include:

- Docker
- Kubernetes
- Amazon SageMaker

because modern AI systems must operate in:

High-Scale, Low-Latency, Production Environments

Deploying AI models into production is only the beginning of operational AI engineering.

As AI applications scale to:

- Millions of users
- Billions of requests
- Real-time inference workloads
- Multi-modal systems
- GPU-intensive pipelines

organizations face a critical challenge:

How can AI systems maintain low latency, high throughput, reliability, and cost efficiency simultaneously?

This chapter explores:

- Throughput optimization
- Latency reduction
- Auto-scaling
- Cost-performance tradeoffs
- Multi-model serving
- Asynchronous processing
- Monitoring and profiling
- Production AI infrastructure optimization

These concepts are foundational to:

- MLOps
- LLMOps
- AIOps
- Cloud-native AI Engineering

Section 1 — Theory & Conceptual Foundation

Chapter 1 — Understanding Latency and Throughput

1.1 What Is Latency?

Latency refers to:

The time taken for an AI system to process a request and return a response.

In real-time AI systems:

- Lower latency improves responsiveness
- Higher latency degrades user experience

Examples of Latency-Sensitive Systems

Application	Why Latency Matters
Chatbots	Users expect immediate responses
Voice assistants	Delays break conversations
Fraud detection	Decisions must occur instantly
Autonomous vehicles	Millisecond delays are dangerous
Live translation	Real-time communication required

1.2 What Is Throughput?

Throughput refers to:

The number of requests an AI system can process within a given time period.

Example:

1000 requests per second

High throughput is essential for:

- Large-scale AI platforms
- Enterprise APIs
- Streaming systems
- Cloud AI services

1.3 Latency vs Throughput

These concepts are related but different.

Metric	Focus
Latency	Speed of one request
Throughput	Total requests handled

Important Insight

Increasing throughput does not always reduce latency.

For example:

- Large LLMs may process many requests simultaneously
- But each individual request may still be slow

Chapter 2 — Best Practices for Reducing Latency

2.1 Why Latency Reduction Matters

The uploaded slides highlight several best practices for reducing latency.

High latency causes:

- Poor user experience
- Higher abandonment rates
- Reduced system responsiveness
- Increased operational inefficiency

2.2 Optimize Auto-Scaling Policies

Auto-scaling automatically:

- Adds instances during high traffic
- Removes instances during low traffic

This prevents:

- Resource starvation
- Overloaded servers
- Performance bottlenecks

Horizontal Auto-Scaling Example

Traffic Spike
↓
New Instances Added
↓
Load Distributed
↓
Latency Reduced

Why Auto-Scaling Matters in AI

AI workloads fluctuate dramatically.

Examples:

- Viral chatbot traffic
- Live streaming spikes
- Enterprise business hours

Without auto-scaling:

- Systems become overloaded
- Latency increases sharply

2.3 Use Model Optimization Techniques

Model optimization reduces:

- Compute requirements
- Memory usage
- Inference latency

Common Optimization Techniques

Technique	Purpose
Quantization	Reduce numerical precision
Pruning	Remove unnecessary weights
Distillation	Compress large models
Tensor optimization	Accelerate matrix operations

Quantization Example

Instead of:

32-bit floating point

models may use:

8-bit integers

reducing:

- Memory usage
- GPU requirements
- Inference cost

2.4 Employ Asynchronous Processing

Asynchronous systems process tasks:

Without blocking execution flow.

This improves:

- Concurrency
- Throughput
- Responsiveness

Synchronous vs Asynchronous Systems

Synchronous

Request → Wait → Response

Asynchronous

Request → Continue Processing
↓
Response Later

Why Async Matters in AI APIs

AI inference often involves:

- GPU waiting
- Network calls
- Vector database queries
- External APIs

Async processing improves utilization.

2.5 Monitor and Profile Systems

The uploaded material highlights:

Monitoring and Profiling

as critical optimization practices.

Monitoring Tracks

- Latency
- GPU utilization
- Memory usage
- API throughput
- Error rates

Profiling Identifies

- Bottlenecks

- Slow functions
- Expensive operations
- Inefficient pipelines

Chapter 3 — Throughput Optimization

3.1 Scaling Through Multiple Instances

The uploaded diagrams show multiple inference instances handling requests simultaneously.

This demonstrates:

Horizontal Scaling

Horizontal Scaling

Instead of:

One Large Server

systems deploy:

Many Parallel Instances

This increases:

- Throughput
- Availability
- Fault tolerance

Example AI Inference Cluster

Load Balancer
↓
Instance 1
Instance 2
Instance 3
Instance 4

Requests distribute across instances.

3.2 Multi-Model Serving

The uploaded diagrams also show:

- Text generation
- Image generation

running inside the same instance.

This is called:

Multi-model serving

Why Multi-Model Serving Matters

Benefits include:

- Reduced infrastructure cost
- Better GPU utilization
- Consolidated deployment

Common in:

- Foundation model platforms
- Multi-modal AI systems

Example

One inference server may host:

- Text embedding model
- Chat model
- Image classifier
- Speech model

simultaneously.

Chapter 4 — Cost Optimization in AI Infrastructure

4.1 Why AI Infrastructure Is Expensive

Modern AI systems consume:

- GPUs
- High-memory servers
- Distributed storage
- High-speed networking

LLM systems are especially expensive.

Major Cost Drivers

Resource	Cost Impact
GPUs	Extremely expensive
Large models	High VRAM consumption
Scaling instances	More infrastructure
Data transfer	Network expenses
Real-time serving	Continuous compute usage

4.2 Scaling vs Cost Tradeoff

The uploaded slides illustrate a key operational tradeoff.

Increased Scaling

Benefits:

- Lower latency
- Better user experience
- Improved availability

But:

- Infrastructure cost increases

Reduced Scaling

Benefits:

- Lower operational cost

But:

- Increased latency
- Poorer responsiveness
- Queue buildup

Core Production Engineering Challenge

Organizations constantly balance:

Performance ↔ Cost

4.3 Cost Optimization Strategies

Modern AI systems reduce costs through:

- Spot instances
- Quantized models
- Dynamic batching
- Serverless inference
- Multi-model endpoints
- GPU sharing

Example: Dynamic Batching

Instead of processing:

1 request at a time

systems combine requests into:

Mini-batches

improving GPU efficiency.

Chapter 5 — Docker Revisited for AI Optimization

5.1 Why Docker Remains Critical

The uploaded Docker workflow revisits the deployment lifecycle:

Dockerfile
↓
Build Image
↓
Run Container

Why Containers Matter for Scaling

Containers provide:

- Reproducibility
- Isolation
- Rapid deployment
- Scalability
- Infrastructure portability

Docker in AI Production Systems

Containers commonly package:

- Inference APIs
- Vector databases
- Model servers
- Monitoring tools
- GPU runtimes

Chapter 6 — Monitoring and Profiling AI Systems

6.1 Why Monitoring Is Essential

AI systems are dynamic.

Performance changes due to:

- Traffic spikes
- Drift
- GPU contention
- Model changes
- Infrastructure failures

Monitoring enables:

Operational Visibility

6.2 Key Metrics to Monitor

Metric	Why It Matters
Latency	User responsiveness
Throughput	System capacity
GPU utilization	Resource efficiency
Memory usage	Stability
Error rates	Reliability
Token usage	LLM cost management

6.3 AI Profiling

Profiling identifies:

- Slow inference stages
- Bottlenecks
- Expensive computations

Example Bottlenecks

Bottleneck	Problem
Token generation	Sequential LLM decoding
Network transfer	Large payload latency
CPU preprocessing	Slow input pipelines
GPU underutilization	Inefficient batching

Chapter 7 — Production Scaling Architectures

7.1 Typical AI Scaling Architecture

Modern AI systems often use:

User
↓
CDN
↓
Load Balancer
↓
API Gateway
↓
Inference Cluster
↓
GPU Nodes
↓
Monitoring Systems

Why Layered Architecture Matters

Each layer handles a different responsibility.

Layer	Responsibility
CDN	Faster content delivery
Load balancer	Traffic distribution
API gateway	Authentication/routing

Layer	Responsibility
Inference cluster	Model serving
Monitoring	Observability

7.2 GPU Clusters

Large AI systems often deploy:

- Multi-GPU clusters
- Distributed inference nodes
- Tensor-parallel architectures

This enables serving:

- Large Language Models
- Multi-modal models
- Real-time AI systems

Section 2 — Practical Implementation & Coding Walkthrough

Chapter 8 — Building Optimized AI Inference APIs

8.1 Installing Dependencies

Cell 1 — Install Required Libraries

```
pip install fastapi uvicorn transformers torch asyncio
```

Detailed Explanation

This installs:

- API framework
- Transformer inference tools
- Deep learning runtime
- Async support

8.2 Create FastAPI Inference API

Cell 2 — Basic AI API

```
from fastapi import FastAPI
from transformers import pipeline

app = FastAPI()

classifier = pipeline(
   "sentiment-analysis"
)
```

Detailed Explanation

This initializes:

- FastAPI backend
- Hugging Face pipeline
- Sentiment model

8.3 Create Async Inference Endpoint

Cell 3 — Asynchronous Endpoint

```
@app.post("/predict")
async def predict(text: str):

   result = classifier(text)

   return {
      "prediction": result
   }
```

Why Async Endpoints Matter

Async APIs:

- Handle concurrent requests better

- Improve scalability
- Prevent blocking operations

This is critical for:

- Chatbots
- LLM APIs
- Real-time inference

8.4 Add Request Timing

Cell 4 — Measure Latency

```
import time

start = time.time()

result = classifier(text)

end = time.time()

latency = end - start
```

Detailed Explanation

This measures:

Inference Latency

which is one of the most important production metrics.

8.5 Add Logging

Cell 5 — Monitor Requests

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info(
```

```
    f"Inference latency: {latency}"
)
```

Detailed Explanation

Logging enables:

- Observability
- Performance tracking
- Production debugging

8.6 Dockerize Optimized API

Cell 6 — Dockerfile

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD [
  "uvicorn",
  "app.main:app",
  "--host",
  "0.0.0.0",
  "--port",
  "8000"
]
```

Why Docker Helps Performance Engineering

Containers simplify:

- Auto-scaling
- Replication

- Deployment consistency

8.7 Kubernetes Horizontal Scaling

Cell 7 — Kubernetes Deployment

```
apiVersion: apps/v1
kind: Deployment

metadata:
 name: inference-api

spec:
 replicas: 4

 selector:
  matchLabels:
   app: inference-api

 template:
  metadata:
   labels:
    app: inference-api

  spec:
   containers:
   - name: inference-container
    image: ai-inference-app
```

Detailed Explanation

This deployment:

- Creates four replicas
- Improves throughput
- Supports high traffic loads

8.8 Kubernetes Auto-Scaling

Cell 8 — Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
 name: inference-hpa

spec:
 scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: inference-api

 minReplicas: 2
 maxReplicas: 10

 metrics:
 - type: Resource
  resource:
   name: cpu
   target:
    type: Utilization
    averageUtilization: 70
```

Detailed Explanation

This automatically:

- Adds pods during high load
- Removes pods during low traffic

Benefits:

- Lower latency
- Better scalability
- Cost optimization

8.9 Production AI Optimization Pipeline

Modern AI serving systems often use:

User Request
↓
Load Balancer
↓
Async API Layer
↓
Dynamic Batching
↓
GPU Inference
↓
Monitoring & Profiling
↓
Response

This architecture supports:

- High throughput
- Low latency
- Scalable AI serving

Final Takeaways

Modern AI deployment is fundamentally:

Distributed Systems Engineering

Production AI systems require:

- Latency optimization
- Throughput scaling
- GPU orchestration
- Cost management
- Async processing
- Dynamic batching
- Monitoring
- Auto-scaling

Key operational goals include:

- Lower latency

- Higher throughput
- Better reliability
- Reduced infrastructure cost

because modern AI systems must support:

Massive-Scale, Real-Time, Business-Critical Workloads